Clipping 1

A. JEEWA

What will be covered?

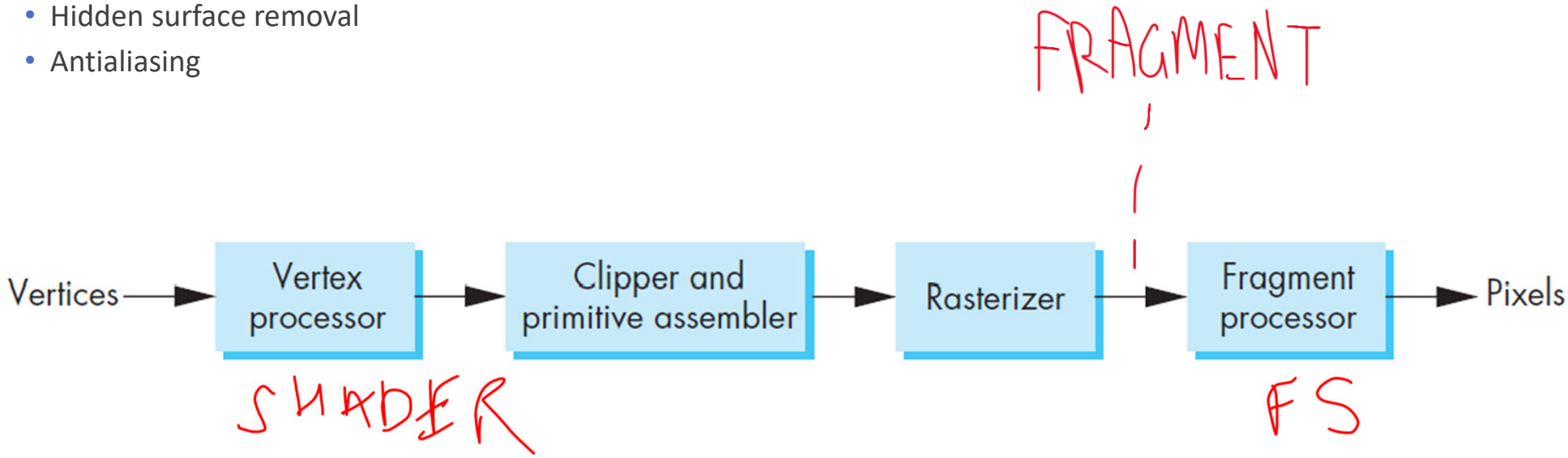
- Clipping
- Rasterization (Scan conversion)
- Hidden Surface Removal

Overview

- At the end of the geometric pipeline, vertices have been assembled into primitives
- Primitives that are outside the view frustum must be clipped out
  - Algorithms are based on representing primitives by lists of vertices
- We must find which pixels can be affected by each primitive. This involves:
  - Fragment generation
  - Rasterization or scan conversion

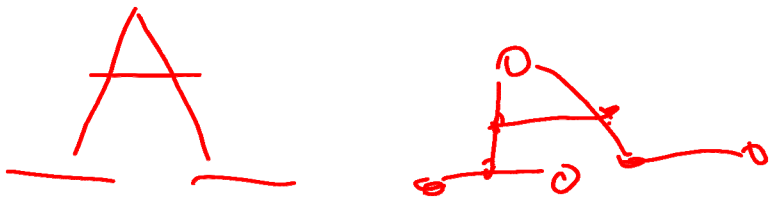
Required Tasks

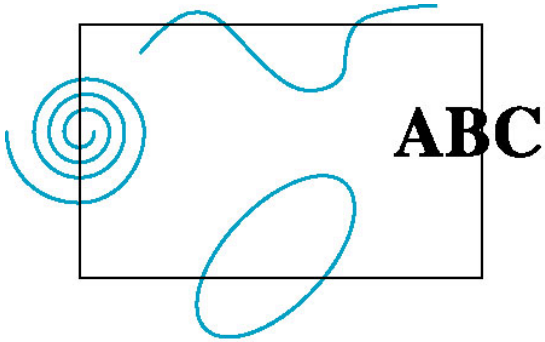
- Clipping
- Rasterization or scan conversion
- Some tasks deferred until fragment processing
  - Hidden surface removal
  - Antialiasing



Clipping

- Restricting or trimming portions of objects that lie outside a defined viewing area or clipping volume
- 2D clipping is done against clipping window
- 3D clipping is done against clipping volume
- Clipping is:
  - Easier for line segments and polygons, and
  - More challenging for curves and text
  - Solution: Convert them to lines and polygons first

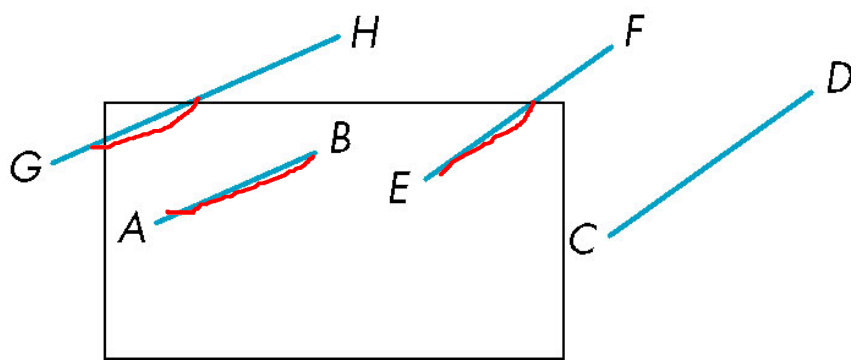




Clipping 2D Line Segments

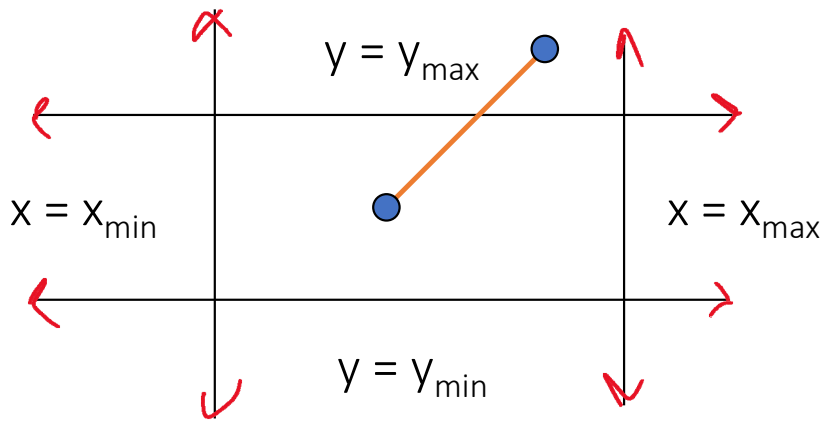
- Brute force approach: compute intersections with all sides of clipping window
- Inefficient: will have many expensive floating point operations for every intersection, especially multiplication and division.

$m = \frac{\Delta y}{\Delta x}$



Cohen-Sutherland Algorithm

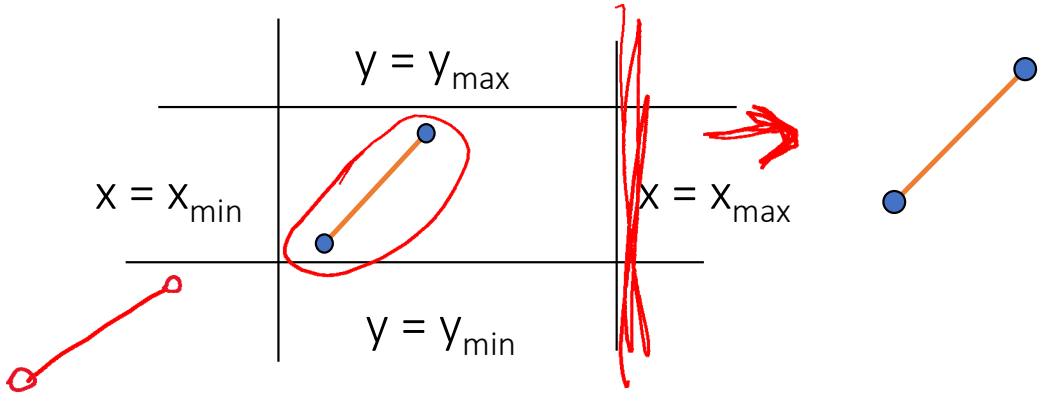
- Idea: eliminate as many cases as possible without computing intersections
- Start with four lines that determine the sides of the clipping window
- The lines are extended to infinity on all ends, thus breaking up space into the nine regions shown below:



The Cases

Case 1: both endpoints of a line segment are inside all four (clipping) lines/ inside the clipping window

- Draw (accept) line segment as is



Case 2: both endpoints are outside all clipping boundaries and on the same side of a line

- Discard (reject) the line segment

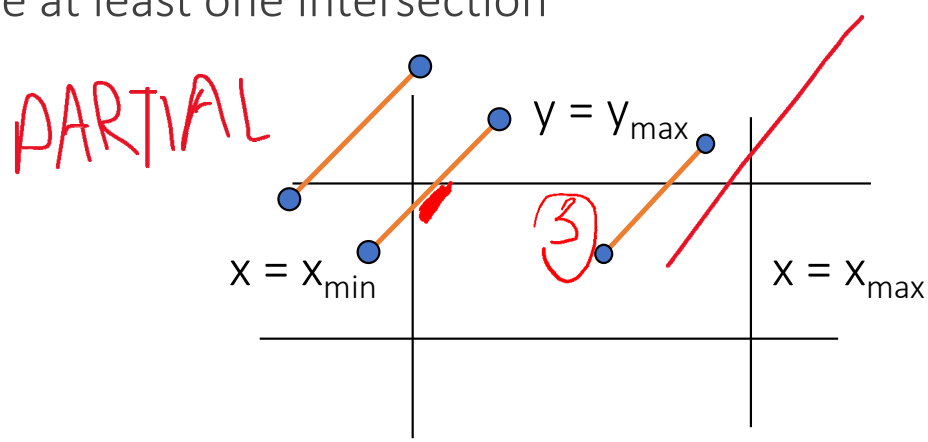
The Cases

Case 3: One endpoint inside the window, one outside

- Must calculate at least one intersection

Case 4: Both endpoints outside

- May have part inside
- Must calculate at least one intersection

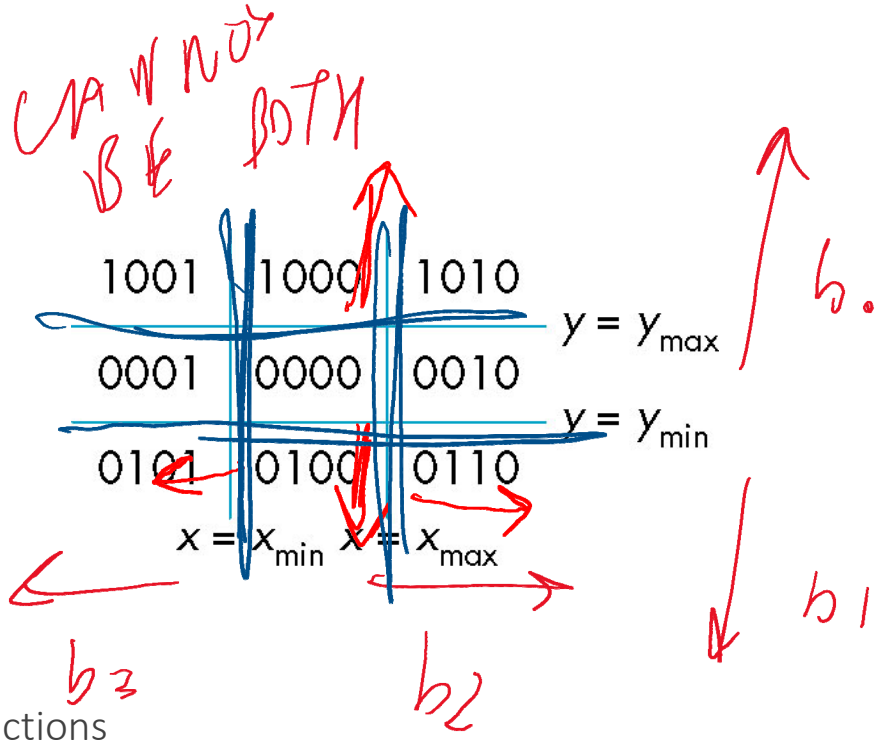


Defining Outcodes

- For each endpoint, define an outcode

$b_0 b_1 b_2 b_3$

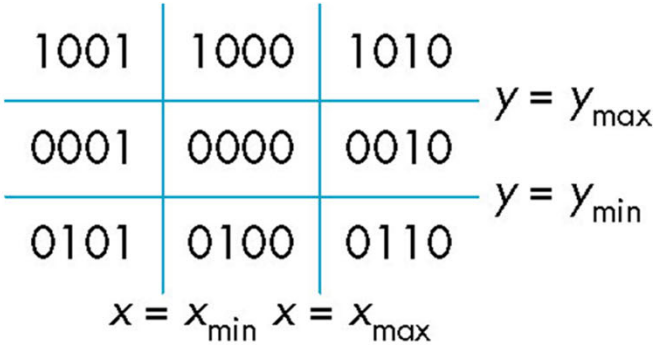
b_0 = 1 if $y > y_{max}$, 0 otherwise
b_1 = 1 if $y < y_{min}$, 0 otherwise
b_2 = 1 if $x > x_{max}$, 0 otherwise
b_3 = 1 if $x < x_{min}$, 0 otherwise

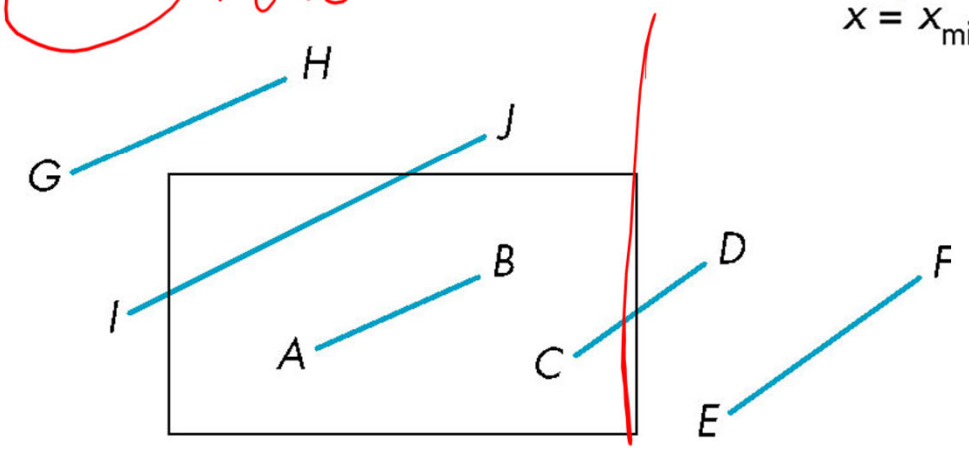


- Outcodes divide space into 9 regions
- Computation of outcode requires at most 4 subtractions

Using Outcodes

- Consider the cases below
- Case 1 (both endpoints of a line segment are inside all four lines)
- AB: outcode(A) = outcode(B) = 0 0000
  - **Accept line segment**

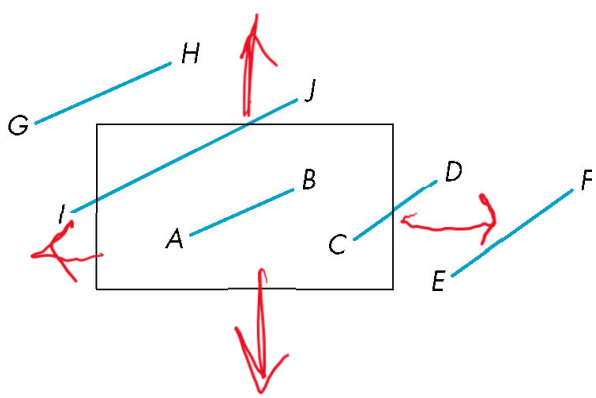




Using Outcodes

E 0010
F 0010

- Case 2 (both endpoints are outside all lines and on the same side of a line)
- EF: outcode(E) logically ANDed with outcode(F) (bitwise) $\neq 0$
  - Both outcodes have a 1 bit in the same place
  - Line segment is outside of corresponding side of clipping window
  - **Reject Line Segment**



Recall: Logical AND

x	y	xy
0	0	0
0	1	0
1	0	0
1	1	1

1001	1000	1010	$y = y_{max}$
0001	0000	0010	$y = y_{min}$
0101	0100	0110	
$x = x_{min}$	$x = x_{max}$		

Using Outcodes

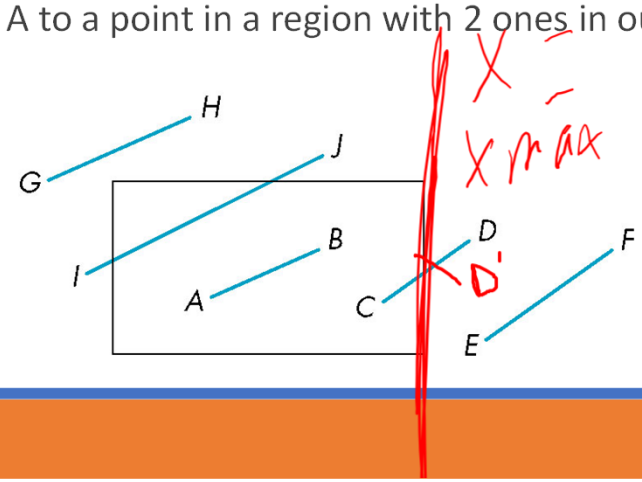
1001	1000	1010
0001	0000	0010
0101	0100	0110

$y = y_{max}$

$y = y_{min}$

$x = x_{min}$ $x = x_{max}$

- Case 3 (One endpoint inside the window, one outside)
- CD: outcode (C) = 0, outcode(D) ≠ 0
  - Compute intersection
  - Location of 1 in outcode(D) determines which edge to intersect with
  - Calculate Intersection
  - Replace the Outside Endpoint CD'
  - REPEAT → TILL CASE 1/2
  - Note if there were a segment from A to a point in a region with 2 ones in outcode, we might have to do two intersections



C = 0000

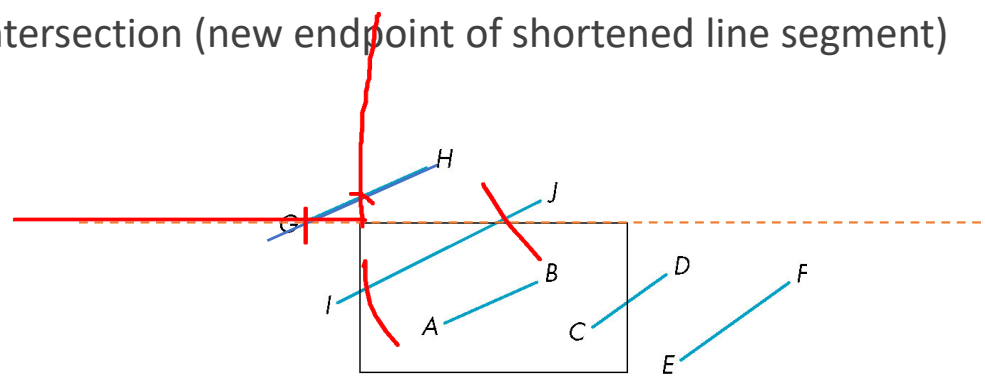
D = 0010

Using Outcodes

G 0001 G ∩ H 0000

H 1000

- Case 4 (Both outside)
- GH and IJ: same outcodes, neither zero but logical AND yields zero
- Shorten line segment by intersecting with one of sides of window
- Compute outcode of intersection (new endpoint of shortened line segment)
- Re-execute algorithm

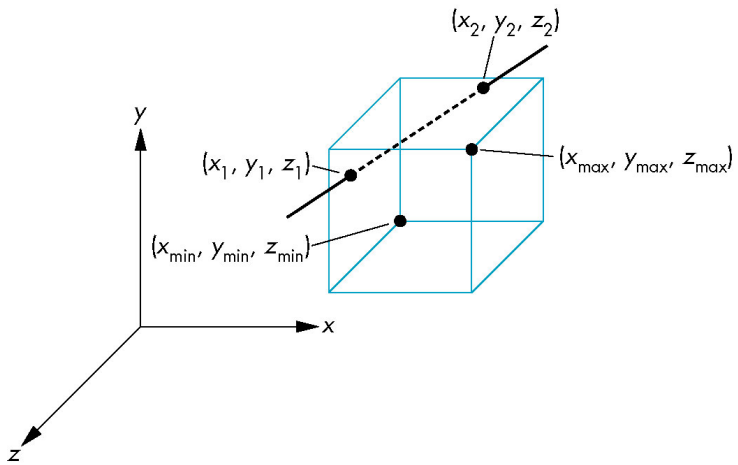


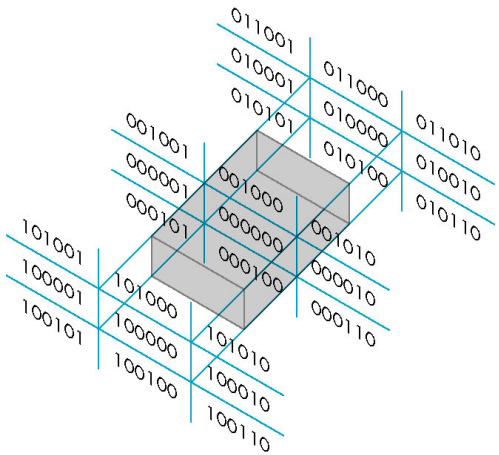
Cohen-Sutherland Algorithm: Efficiency

- In many applications, the clipping window is small relative to the size of the entire scene
  - Most line segments are outside one or more sides of the window and can be eliminated based on their outcodes
- Inefficiency when code has to be re-executed for line segments that must be shortened in more than one step

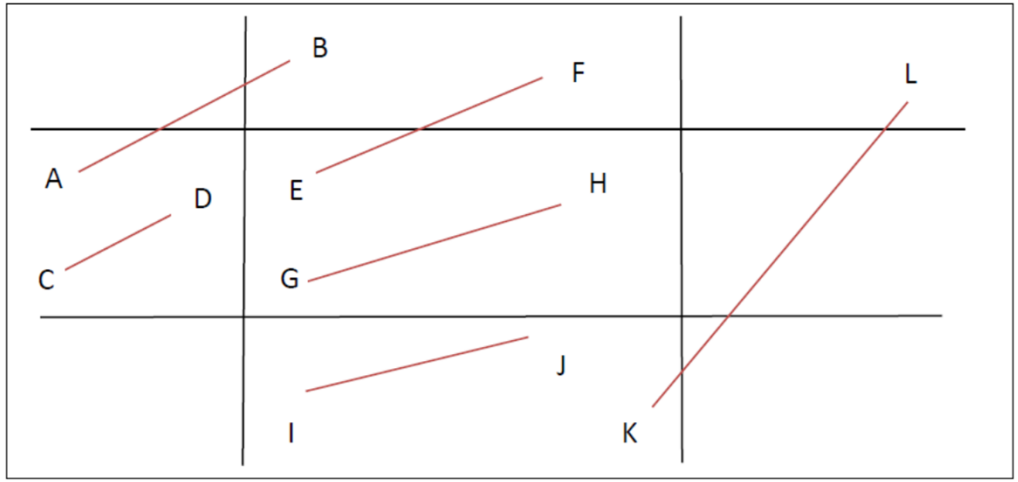
Cohen-Sutherland in 3D

- Use 6-bit outcodes
- When needed, clip line segment against planes





Cohen-Sutherland - Example



a. Give the Cohen-Sutherland outcodes for the points for the following lines:
   i. AB
   ii. EF
   iii. KL

b. List all the lines whose intersections would NOT be computed, if the Cohen-Sutherland Clipping algorithm was to be used for line-segment clipping.